



Pahari — Future Feature Backlog & Specifications

This document defines the product specifications, clinical problems, implementation flows, and technical architectures for upcoming Pahari features. Each feature is classified by its **Production Deployment Preference** to highlight how to build it:

- **[LOCAL PREFERRED – ALGORITHM]** : Built using client-side JavaScript or server-side Python algorithms (e.g., regex, constraint solvers, static JSON lookups). Needs 0% AI models and 0% cloud APIs.
 - **[LOCAL PREFERRED – MACHINE LEARNING]** : Built using lightweight, open-source ML models (e.g., OpenCV, ONNX Runtime Web, Tesseract, Whisper.cpp, or local quantized LLMs like Llama-3.2-1B) running on browser WebGPU or cheap CPU cloud servers.
 - **[REQUIRES CLOUD API]** : Must call external web APIs because of massive database size (e.g., openFDA, RxNorm, Google Places) or complex proprietary models (e.g., multimodal Gemini Flash).
-

💎 C. "Killer Features" (Viral Acquisition & High Retention)

1. 🧠 The "Context-Aware" Personal AI Doctor (Pocket Clinician)

- **Classification:** **[LOCAL PREFERRED – MACHINE LEARNING]**
 - **Why Local is Preferred:** Patient health queries contain highly sensitive, private medical data. Running this locally via the browser's WebGPU or a quantized server CPU guarantees compliance with data privacy regulations (like HIPAA) and avoids expensive cloud-token subscription fees.
 - **The Core Problem:** Standard medical chatbots (like ChatGPT) are stateless and generic. They do not know the patient's age, allergies, active medication cabinet, or latest blood reports. The patient is forced to copy-paste their entire medical history every time they ask a question, or risk receiving dangerous, generic advice.
 - **The Solution & User Flow:** A conversational chatbot interface operating on the home screen. It has direct context access to the patient's local profile databases (`localStorage` cabinet lists, allergies list, recent triage logs, and parsed lab test trendlines). When the user asks a question, a local script injects this localized profile into the model's prompt.
 - *User Flow:* The user asks, "I have a sudden headache. Can I take an Advil?" The local model checks the active cabinet, notices they are taking a blood thinner, and replies, "No, John. You are currently taking Warfarin. Advil (Ibuprofen) interacts with Warfarin and increases your internal bleeding risk. Try taking Acetaminophen (Tylenol) instead, which is safe for you."
 - **Tech Stack:**
 - *Cloud Fallback:* LangChain/LlamaIndex, Google Gemini Pro API.
 - *Local Build:* **WebLLM** (in-browser WebGPU execution of `Llama-3.2-1B-Instruct` or `Gemma-2-2B-it`) or a backend **Ollama/llama.cpp** endpoint running a quantized 3B model.
-

2. 📄 The "Doctor's Handwriting" Decipherer (Cursive Clinical OCR)

- **Classification:** **[REQUIRES CLOUD API]** (with **[LOCAL PREFERRED – MACHINE LEARNING]** fallback)
- **Why API is Required:** Cursive handwriting deciphering is extremely difficult. It requires high-parameter multimodal vision models. While standard printed OCR can run locally, reading "chicken scratch" doctor handwriting requires cloud-scale neural networks.
- **The Core Problem:** Handwritten prescriptions are notoriously illegible. Patients leave consultations with no idea what medication was prescribed, what the dosage is, or how to spell it, making them entirely dependent on a pharmacist to interpret the writing.
- **The Solution & User Flow:** A dedicated scanning module. The user snaps a photo of a handwritten doctor's note or prescription. The image is preprocessed and sent to a multimodal AI model prompted with handwriting guidelines and medical dictionaries.

- *User Flow*: The user uploads a photo of cursive handwriting. The AI deciphers it and outputs clean digital text: "Rx: Metformin 500mg - 1 tab twice daily after meals (BD PC)", alongside clickable links to drug profiles.
 - **Tech Stack**:
 - *Primary API*: Google Gemini Flash API (Multimodal OCR).
 - *Local Build*: OpenCV (for contrast enhancement, deskewing) + PyTorch hosting a local **TrOCR** (Transformer OCR) model on the backend, paired with a fuzzy spell-checker against a local drug name dictionary.
-

3. 📄 "Paper-to-Dashboard" Smart Health Locker & Trendline Generator

- **Classification**: [LOCAL PREFERRED – ALGORITHM] (paired with local OCR)
 - **Why Local is Preferred**: Lab reports are highly standardized. You do not need expensive cloud APIs to extract coordinates from table cells; standard image parsing algorithms can extract this data locally.
 - **The Core Problem**: Chronic disease patients (managing diabetes, hypertension, thyroid conditions) accumulate folders of physical lab reports from different diagnostics clinics. Doctors rarely have time to look through years of paper records, meaning subtle clinical trend changes are missed.
 - **The Solution & User Flow**: A digital health repository. The user takes a photo of a physical lab report (e.g. blood test, lipid panel). The local OCR engine parses the document, matches clinical biomarkers (e.g. HbA1c, Cholesterol levels, TSH), normalizes units, and plots them on an interactive trendline graph.
 - *User Flow*: The user uploads three separate blood tests from different dates. Prahari generates an interactive line chart showing their blood sugar levels over time, with color-coded warning zones.
 - **Tech Stack**:
 - *Primary Local*: OpenCV (applying morphological dilation to isolate table grid lines), **Tesseract OCR (pytesseract)** to extract text inside grid boundaries, regex mapping, and Chart.js/Recharts for rendering.
-

4. 🏠 Government Subsidy & 90% Cost-Saver Finder (Ayushman Bharat / Jan Aushadhi Matcher)

- **Classification**: [LOCAL PREFERRED – ALGORITHM]
 - **Why Local is Preferred**: This is database-driven logic. It only requires checking a local dataset matching active salt names to government generic equivalents.
 - **The Core Problem**: Medications are a heavy financial burden. In countries like India, the government sells generic medicines at 90% cheaper rates via *Jan Aushadhi Kendras*, and covers free hospital treatments under *Ayushman Bharat (PM-JAY)*. However, patients have no easy way to check generic availability or locate covered hospitals.
 - **The Solution & User Flow**: When a user scans or searches for a brand-name medication, Prahari automatically runs a query matching the active salts against the government generic database. It displays the price difference and maps the nearest generic distributor.
 - *User Flow*: Searching for "Janumet 50/500" displays: "Brand Price: ₹220 | Government Generic Price: ₹18 (91% Savings). Available at Jan Aushadhi Kendra (240 meters away)" with a Leaflet map route.
 - **Tech Stack**: SQLite/PostgreSQL lookup tables, **Levenshtein Distance fuzzy-matching algorithm** in Python/JS, Leaflet Map integration, geolocation distance calculator.
-

5. 🏠 AI Visual Pill Identifier & Verification ("Pill Scan")

- **Classification**: [LOCAL PREFERRED – MACHINE LEARNING]

- **Why Local is Preferred:** Running this client-side in the browser using WebGL or WebAssembly allows for instant, real-time feedback through the camera preview without uploading heavy raw video frames to a cloud server.
 - **The Problem:** Elderly patients taking multiple daily pills often mix up loose pills in their weekly organizers, leading to accidental double-dosing or omissions.
 - **The Solution & User Flow:** A computer vision camera view. The user places their daily pill compartment or a loose pill on a neutral surface and takes a photo. The scanner identifies the pills' color, shape, and imprints, comparing it in real-time against their scheduled daily cabinet prescription.
 - *User Flow:* Snapping a photo of three loose pills returns: "White round pill = Metformin (Correct); Blue capsule = Ibuprofen (Correct); WARNING: Yellow pill = NOT scheduled for today! Please check your prescription."
 - **Tech Stack:**
 - *Local Build:* OpenCV contour analysis (circularity and aspect ratio computation) + **ONNX Runtime Web** executing a lightweight **MobileNetV3** classification model in the browser.
-

6. 📅 The AI Medication Schedule Optimizer (Chronotherapy Engine)

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
 - **Why Local is Preferred:** Schedule optimization is a mathematical constraint problem, not a generative language task. Generating schedules with an LLM introduces severe safety risks (hallucinations). A local, deterministic algorithm guarantees safety.
 - **The Problem:** Patients taking multiple pills often take them all at once. However, drugs interact in the stomach, binding together or blocking absorption (e.g. taking iron with calcium, or blood pressure pills at the wrong hour).
 - **The Solution & User Flow:** An automated calendar organizer. When a user adds drugs to their cabinet, the algorithm automatically calculates the optimal schedule, distributing pill times throughout the day to maximize absorption and eliminate conflicts.
 - *User Flow:* The user enters three medications. The app schedules: "Take Pill A at 8:00 AM (Empty Stomach); Take Pill B at 12:00 PM (With Food); Take Pill C at 8:00 PM (Separate from Calcium)" with notifications.
 - **Tech Stack:** **Constraint Satisfaction Problem (CSP) solver** written in JS or Python, local JSON database mapping drug absorption inhibitors and optimal time blocks.
-

7. 🗨️ The AI Insurance Claim Denials Appeals Copilot

- **Classification:** [LOCAL PREFERRED – MACHINE LEARNING]
 - **Why Local is Preferred:** Drafting formal appeal letters based on medical codes is highly template-driven. It can be accomplished locally using small, focused language models or structured questionnaires.
 - **The Problem:** Insurance companies deny millions of claims using automated rejection algorithms. Reversing rejections requires medical jargon, references to FDA labels, and hours of drafting letters.
 - **The Feature:** The user snaps a photo of their insurance rejection letter. The local engine reads it, parses the rejection code, searches clinical guidelines, and generates a legally and clinically authoritative appeal letter ready to sign and mail.
 - **Tech Stack:**
 - *Local Build:* Quantized **Llama-3.2-3B** running locally, combined with a structured template engine (filling clinical text templates based on the parsed insurance code).
-

8. 🗣️ AI Vocal Biomarker Scan (10-Second Diagnostic)

- **Classification:** [LOCAL PREFERRED – MACHINE LEARNING]

- **Why Local is Preferred:** Audio processing is computationally cheap. Extracting vocal frequencies (FFTs) can be done entirely in the browser using the Web Audio API, and the classification model (e.g. SVM/Random Forest) is under 1MB, meaning zero backend cost.
 - **The Problem:** Early signs of respiratory illnesses (asthma, COPD) or voice disorders are often missed by patients until they escalate.
 - **The Feature:** The user holds a button and speaks a standard phrase for 10 seconds. The app analyzes vocal biomarkers to screen for early signs of respiratory distress.
 - **Tech Stack:** Web Audio API (for audio recording), **Librosa / Scipy.signal** (for extracting Pitch, Jitter, Shimmer, and MFCCs via Fast Fourier Transforms), and a local **Scikit-Learn Random Forest/SVM** classifier model.
-

9. 🚨 Voice-Activated "Guardian Sentinel" & Inactivity Alerts

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
 - **Why Local is Preferred:** This relies entirely on hardware device sensors and background tasks.
 - **The Problem:** Caregivers worry constantly about elderly parents living alone—especially about sudden falls or forgetting critical medications.
 - **The Feature:** If a critical medication reminder is not checked off within 2 hours of its schedule, or if the app detects zero movement on the phone, the app automatically triggers a high-priority SMS alert to the caregiver.
 - **Tech Stack:** Browser DeviceMotion API, LocalStorage reminder schedules, background service workers, and a backend SMS trigger (e.g. Twilio API).
-

10. 🥗 Food-Drug & Diet Safety Guard (Food Contraindications)

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
 - **Why Local is Preferred:** Simple relational database query. No AI required.
 - **The Problem:** Drug-food interactions (e.g., taking thyroid pills with milk, or blood thinners with spinach) are common and dangerous, yet rarely flagged.
 - **The Feature:** Cross-references the user's active "Medicine Cabinet" list against a database of common dietary ingredients, displaying warning flags before meals.
 - **Tech Stack:** Static JSON key-value dictionary mapping active salts to contraindicating food elements.
-

11. 🗣️ Consultation Note Transcriber & Auto-Scheduler

- **Classification:** [LOCAL PREFERRED – MACHINE LEARNING]
 - **Why Local is Preferred:** Recording doctor consultations involves extreme privacy concerns. Uploading raw audio files of private doctor visits to the cloud is high-risk. Running transcription locally on the client's phone ensures absolute privacy.
 - **The Problem:** Patients forget up to 80% of what the doctor said during a hurried consultation.
 - **The Feature:** A voice recorder that records the doctor's consultation, transcribes the speech locally, extracts drug names, dosages, and schedules, and automatically builds the patient's Medicine Cabinet.
 - **Tech Stack:** Web Audio API, **Whisper.cpp / Faster-Whisper** (quantized Whisper-small model, ~140MB, running locally on device/server CPU), and a local SpaCy pipeline for clinical entity parsing.
-

🎨 A. Premium UI/UX & Engagement Upgrades

12. 📅 Daily Medication Scheduler Widget

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
- **Why Local:** React local state, CSS animations, and browser `localStorage`. No server needed.

- **The Solution:** An interactive checklist widget on the homepage using satisfying animations and streak trackers for marking doses as taken.

13. 🗄️ "My Medicine Cabinet" Virtual Drawer

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
- **Why Local:** React Context API, CSS transitions, and local database storage.

14. ✂️ Drug-Drug Interaction Matrix

- **Classification:** [LOCAL PREFERRED – ALGORITHM] (with API fallback)
- **Why Local:** A pairwise grid checker can run client-side using a cached local database of known interactions to avoid spamming external APIs.
- **The Solution:** Interactive comparison panel. Users select two or more medications and see a color-coded safety warning matrix.
- **Tech Stack:** Local caching of RxNorm/openFDA query outputs, paired with client-side grid mapping.

15. 🗺️ Visual Body Map Selector

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
- **Why Local:** Interactive frontend graphics. No server needed.
- **The Solution:** An interactive SVG model of a human body. Users click on the affected area (e.g., Head, Chest) to automatically generate search tags for triage.

16. 💬 Conversational Triage Chatbot

- **Classification:** [REQUIRES CLOUD API] (with [LOCAL PREFERRED – ALGORITHM] fallback)
- **Why API is Required:** Accurate clinical diagnostic questioning requires deep, curated medical knowledge graphs (like Infermedica) that are too large and proprietary to run locally.
- **The Solution:** A friendly AI chat assistant that guides the user through diagnostic questions to evaluate symptom severity.
- **Tech Stack:** React chat interface, **Infermedica API**.

17. 🗺️ Leaflet / OpenStreetMap Inline Map

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
- **Why Local:** Maps can be rendered on the client side without paid Google Maps APIs.
- **The Solution:** Embeds interactive Leaflet maps in the directory search view to display clinic markers.
- **Tech Stack:** Leaflet.js, OpenStreetMap tiles (free alternative to Google Maps SDK).

18. 📱 Simulated QR Appointment Pass

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
- **Why Local:** QR codes are generated using standard mathematical libraries client-side.
- **Tech Stack:** `qrcode.react` library running in the frontend.

🌟 B. Innovative Care Additions

19. 📄 Latin Prescription Abbreviation Decoder

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
- **Why Local:** Latin medical shorthand (BD , TDS , AC) is highly structured and easily parsed using regular expressions.
- **Tech Stack:** Regex dictionary mapping (e.g., `(?i)\bb\.\i\d\b|\bbd\b -> Twice daily`).

20. 🗺️ Caregiver Offline QR Sync

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
- **Why Local:** Offline operations using mathematical compression.
- **The Solution:** Compresses schedules and cabinet lists into a single compact JSON string, encodes it into a QR code, and allows another device to scan and import it offline.
- **Tech Stack:** JSON compression (`lz-string`), QR code generation.

21. 🚑 Triage First-Aid Guides

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
- **Why Local:** Simple static card rendering.
- **Tech Stack:** Local HTML/Markdown list of first-aid procedures.

22. 📷 Client-Side Barcode Scanner

- **Classification:** [LOCAL PREFERRED – ALGORITHM] (processing) / [REQUIRES CLOUD API] (lookup)
- **Why API is Required:** The barcode reading is done locally on the device camera, but matching the barcode number (UPC/NDC) to a drug profile requires looking up a massive, global database of pharmaceutical barcode registrations.
- **Tech Stack:** `html5-qrcode` (local frame processing) + openFDA Barcode Lookup API.

23. 🌸 Gemini Flash Multimodal OCR Upgrade

- **Classification:** [REQUIRES CLOUD API]
- **Why API is Required:** Gemini Flash is a massive cloud model that does not run on local client devices or cheap VM CPUs.
- **Tech Stack:** Google Gemini Flash API.

24. 💬 WhatsApp Pharmacy Reorder Link

- **Classification:** [LOCAL PREFERRED – ALGORITHM]
- **Why Local:** URL formatting only.
- **Tech Stack:** JavaScript WhatsApp API link generation (`wa.me/phone/?text=...`).

25. 🍌 Fuzzy Generic Salt / Substitute Finder

- **Classification:** [REQUIRES CLOUD API]
- **Why API is Required:** Finding equivalent brands and salts requires looking up the latest national pharmaceutical databases, which are updated daily and are too large to package inside a local application.
- **Tech Stack:** openFDA API, active salt mapping database.